

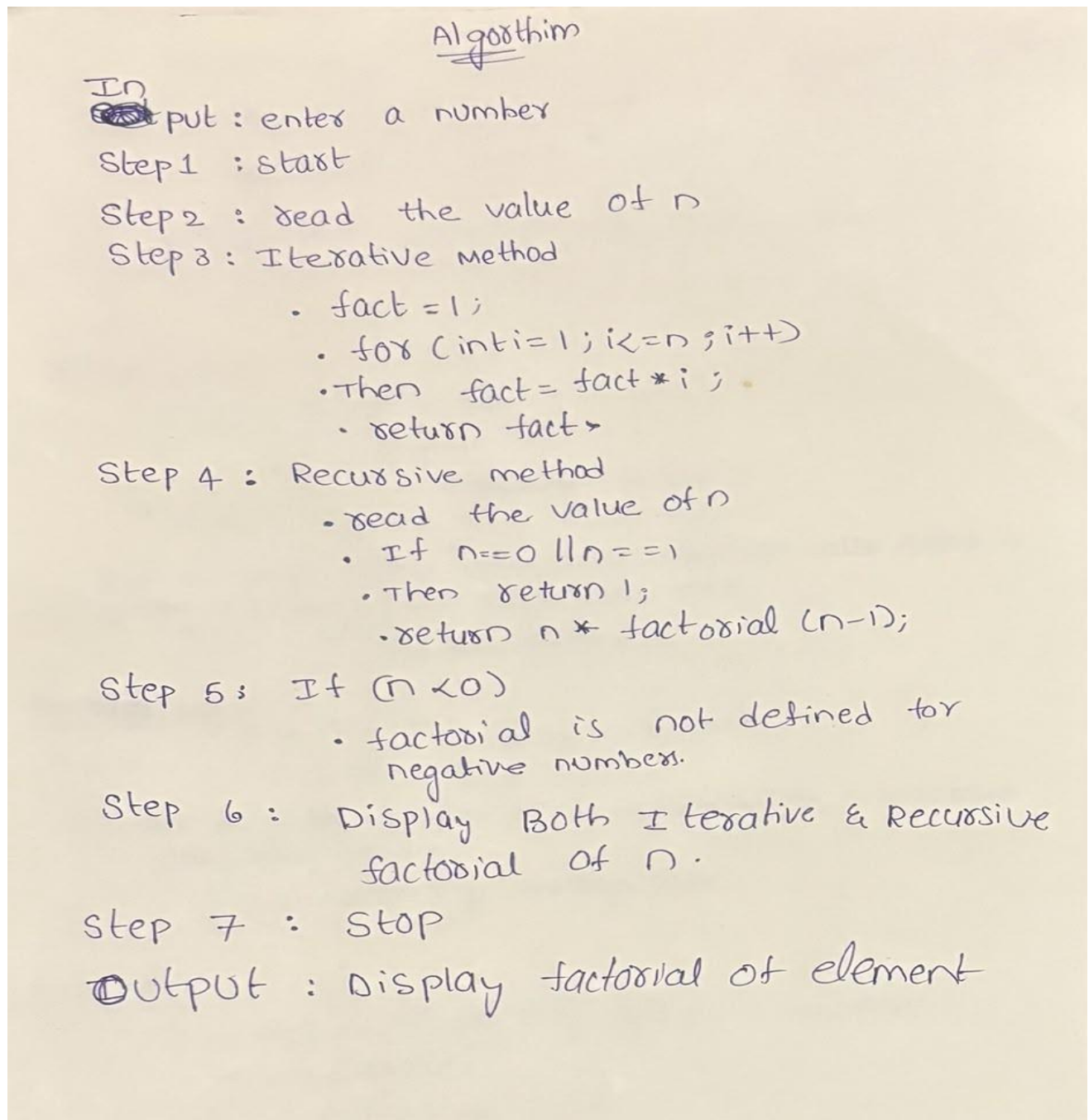
Practical:4

Implementation and Time analysis of factorial program using iterative and recursive method

/08/2026

Aim: Implementation and Time analysis of factorial program using iterative and recursive method

Algorithm:



Algorithm

Input : enter a number

Step 1 : start

Step 2 : read the value of n

Step 3 : Iterative method

- fact = 1;
- for (int i = 1; i <= n; i++)
- Then fact = fact * i;
- return fact;

Step 4 : Recursive method

- read the value of n
- If $n = 0$ || $n = 1$
- Then return 1;
- return $n * \text{factorial}(n-1)$;

Step 5 : If $(n < 0)$

- factorial is not defined for negative numbers.

Step 6 : Display Both Iterative & Recursive factorial of n.

Step 7 : STOP

Output : Display factorial of element

Program:

```
#include <iostream>
using namespace std;
```

```
int factorialIterative(int n)
```

```
{
    int fact = 1;

    for (int i = 1; i <= n; i++)
    {
        fact = fact * i;
    }

    return fact;
}
```

```
int factorialRecursive(int n)
```

```
{
    if (n == 0 || n == 1)
        return 1;

    return n * factorialRecursive(n - 1);
}
```

```
int main()
{
    int n;

    cout << "Enter a number: ";
    cin >> n;

    if (n < 0)
    {
        cout << "Factorial is not defined for negative numbers.";
        return 0;
    }
    int iterativeResult = factorialIterative(n);
    int recursiveResult = factorialRecursive(n);

    cout << "\nFactorial using Iterative Method = "<< iterativeResult ;

    cout << "\nFactorial using Recursive Method = "<< recursiveResult;

    return 0;
}
```

Output:

```

C:\Users\Admin\OneDrive\Pic x + v
Enter a number: 6

Factorial using Iterative Method = 720
Factorial using Recursive Method = 720
-----
Process exited after 14.29 seconds with return value 0
Press any key to continue . . .

```

Time Complexity:

Recursive Method

```

int factorial(n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n-1);
}

```

Worst case

$$T(n) = n + 1$$

Ignoring constant

$$T(n) = n$$

for a large value of n , the function calls itself n times before reaching the base case

$T(n) = O(n)$ worst case

Average case

$$T(n) = n + 1$$

Ignoring constant

$$= n$$

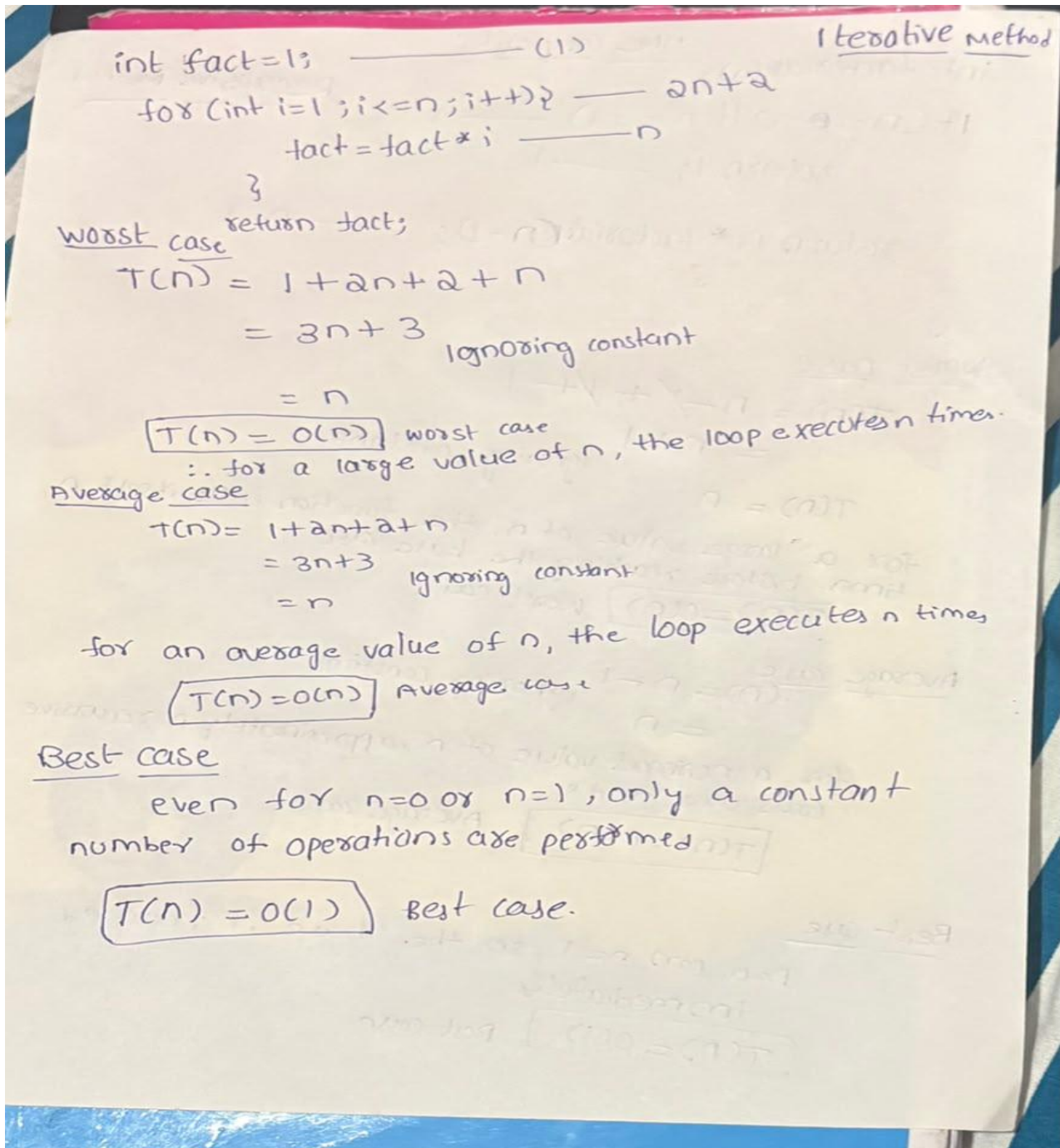
for a normal value of n , approximately n recursive calls are made

$T(n) = O(n)$ Average case

Best case

$n=0$ or $n=1$, so the base is reached immediately

$T(n) = O(1)$ Best case



Conclusion:

Both iterative and recursive methods correctly calculate factorial. Both take $O(n)$ time, but the iterative method uses less memory ($O(1)$) compared to recursion ($O(n)$). Thus, iteration is more memory-efficient, while recursion is easier to understand conceptually.